Seguimiento de experimentos

En el proyecto vas a probar muchos modelos. Si no lo anotas, dentro de una semana nadie sabrá cuál funcionó ni por qué.

Reproducibilidad: semillas y versiones

- Muchos pasos usan azar (partir datos, inicializar pesos, submuestrear): sin **semilla**, cada ejecución da un resultado distinto.
- Distintas **versiones** de las librerías también cambian resultados.

```
def fijar_semillas(semilla):
    random.seed(semilla); np.random.seed(semilla)
    os.environ["PYTHONHASHSEED"] = str(semilla)
    tf.random.set_seed(semilla)                  # si usas redes
```

Regla del equipo: **una función, una semilla**, guardada en `utilidades.py`.

El registro: una fila por prueba

Cada vez que entrenas y evalúas, anotas: **cuándo, qué modelo, con qué ajustes, con qué datos, qué métrica y cuánto tardó**.

```
registrar(responsable, modelo, hiperparametros, datos_hash, semilla,
          metrica, valor_cv, desv_cv, tiempo_s, notas="")
```

- Un CSV en el repositorio (`experimentos/registro.csv`): se abre con Excel y se versiona con Git.
- Plantilla: `registro_experimentos_plantilla.csv`.
- **Cada integrante** debe tener al menos una fila a su nombre.

Baseline y modelos con validación cruzada

```
experimento("Baseline", DummyClassifier(strategy="prior"), {})
experimento("RandomForest", RandomForestClassifier(n_estimators=200, min_samples_leaf=5,
            random_state=SEMILLA, n_jobs=2), {"n_estimators": 200})      # Ejercicio 3
for d in (3, 6, 10):
    experimento(f"RandomForest_prof{d}", ...max_depth=d...)               # Ejercicio 4
```

- `experimento` crea el pipeline, mide con **validación cruzada**, mide el tiempo y **registra**.
- Compara siempre con el *baseline*: ¿cuánto mejora el modelo y a qué coste?

Candidato y evaluación única en test

1. Elige el **candidato** solo con criterios de **validación cruzada** y **sin fuga**: mayor `valor_cv` entre las filas **sin** «Fuga» en el nombre.
2. Reentrénalo y evalúalo en el **test, una sola vez** (Ejercicio 5).
3. Anota el resultado en el registro.

> Si usas el test para elegir, ya no es un examen: es un ejercicio que has visto antes.

MLflow (opcional): seguimiento profesional con interfaz web. No es obligatorio: **el CSV cumple la rúbrica**.

Tu proyecto: qué hacer hoy

1. Crea la carpeta `experimentos/` en el repo del equipo.
2. Copia `fijar_semillas`, `registrar` y `experimento` a `utilidades.py`.
3. Registra **≥ 3 modelos + baseline** con **tus** datos.
4. *Commit* con el CSV; cada persona, al menos una fila.

Según el brief: modelos clásicos, red neuronal o LLM/RAG. **Tutoría técnica de 20 minutos por equipo** mañana: anota tus dudas.

Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

CUADERNO

S15_01_seguimiento_experimentos

50 min

Práctica de seguimiento de experimentos

sprints/S15/notebooks/S15_01_seguimiento_experimentos.ipynb

RETO

Reto del sprint

130 min

Proyecto: modelo de referencia y primer modelo mejor. Anota todo en el registro de experimentos

sprints/S15/guia/Reto_Sprint_15.md

RETO

Reto del sprint (sigue)

100 min

Proyecto: prueba modelos (clásico, red o LLM/RAG, según tu brief)

sprints/S15/guia/Reto_Sprint_15.md